ACCESSING DATA: OBJECT RELATIONAL MAPPING

Pascal URSO
M1 Informatique
Université Côte d'Azur

OBJECTIVE

- Application = object-oriented programming language
- Data = SQL rows (or JSON documents)
- How to access (create, read, update, delete) data elements ?
  - Seamlessly
  - With portability
- Object Relational Mapping
  - PHP (Doctrine, Eloquent), Python (Django), Ruby (on Rails Active Records), C#/.NET (Entity Framework)
  - Java : Hibernate (2001), JPA (2006), **Jakarta Persistance (2022)**

JDBC ?

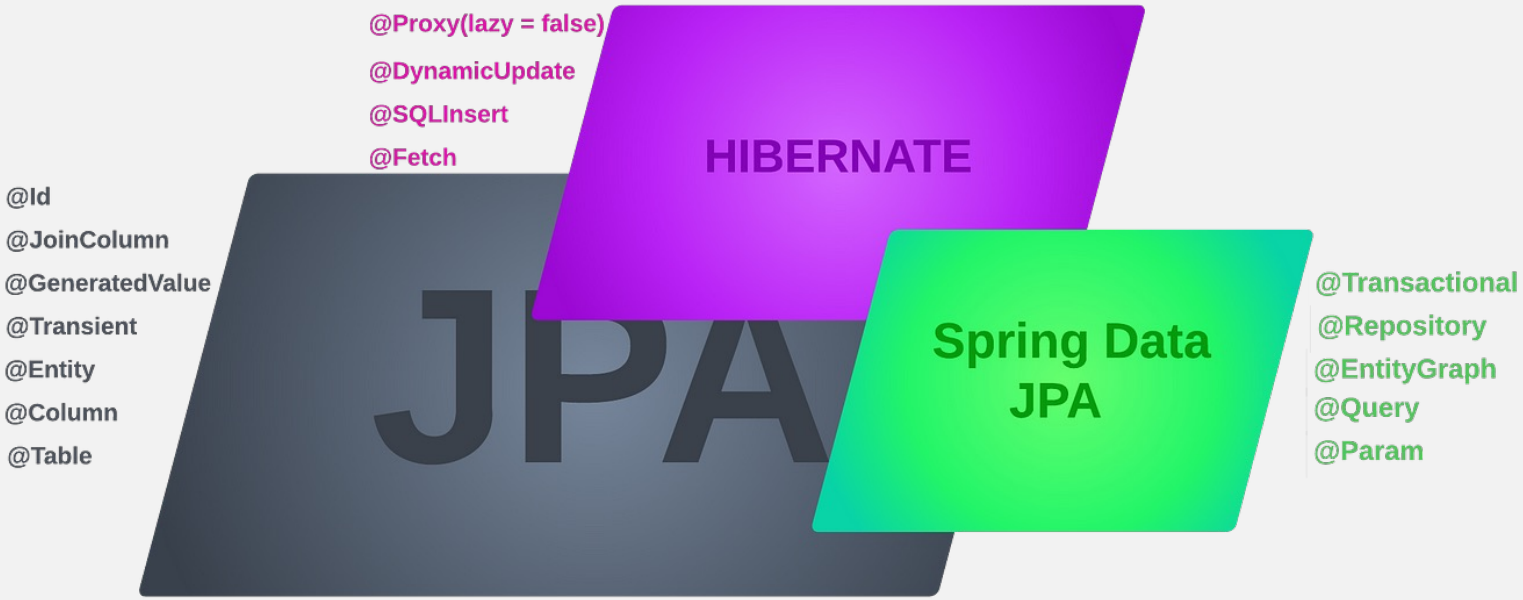
```java
public Customer loadCustomer(long customerId, Connection connection) {
    if (0 == customerId || connection == null) {
        return null;
    }

    PreparedStatement statement = null;
    try {
        // préparation de la requête à la base de données
        statement = connection.prepareStatement(
                " SELECT CUSTOMER_ID, CUSTOMER_NAME, CUSTOMER_ADDRESS "
                + " FROM CUSTOMER WHERE CUSTOMER_ID = ? ");
        statement.setLong(1, customerId);
        ResultSet resultSet = statement.executeQuery();
        if (resultSet.next()) {
            // transformation des données relationnelles en objet "Customer"
            Customer customer = new Customer();
            int index = 1;
            customer.setCustomerId(resultSet.getLong(index++));
            customer.setCustomerName(resultSet.getString(index++));
            customer.setCustomerAddress(resultSet.getString(index++));
            return customer;
        }
    } catch (final Exception e) {
        log.info("", e);
    } finally {
        // nettoyage
        if (statement != null) {
            try {
                statement.close();
            } catch (final SQLException e) {
                log.info("", e);
            }
        }
    }
    return null;
}
```

- Duplication de code
- Mélange de languages
- Gestion des relations ?
- Gestion des transactions ?

JPA

- Jakarta Persistence API (formally Java Persistence API)
- Part of Jarkata (formally JEE, J2EE, …)
- An application programming interface **specification**
- Several Implementations
  - *Hibernate*
  - EclipseLink
  - OpenJPA (Apache)

FRAMEWORKS


@Proxy(lazy = false)
@DynamicUpdate
@SQLInsert
@Fetch
HIBERNATE
@Id
@JoinColumn
@GeneratedValue
@Transient
@Entity
@Column
@Table
JPA
Spring Data JPA
@Transactional
@Repository
@EntityGraph
@Query
@Param

BASIC PRINCIPLES

- Entity
  - Objects that can be persisted in the database
- Entity relationship
  - One-to-one, one-to-many, many-to-many
- Repositories CRUD
  - Create, read, update, delete
- Query language

ENTITY

ENTITY = MAPPED BEAN

```java
import jakarta.persistence.*;

@Entity
@Table(name = "nobels")
public class Customer {

  @Id
  @GeneratedValue(strategy=GenerationType.AUTO)
  private Long id;
  private String firstName;
  @Column(name = "surname")
  private String lastName;

  public Customer() {}

  public Long getId() {
    return id;
  }

  public String getFirstName() {
    return firstName;
  }

  public String getLastName() {
    return lastName;
  }
```

```
SELECT * FROM nobels;
```

id	firstName	surname
6	Marie	Curie
25	Abert	Einstein
26	Niels	Bohr

BASIC MAPPING

- Optional annotations
  - **@Table**
    - Properties : schema, name, indexes, uniqueConstraints
    - Default name (in Hibernate) : first letter capitalized (can be changed)
  - **@Id :** primary key
  - **@GeneratedValue**
  - **@Column**
    - Properties : name, nullable,, unique, length, precision, columnDefinition (SQL), …
- *Could also be defined using XML configuration file (*<mapping-file> in persistence.xml)

ADVANCED MAPPING

- **@SecondaryTable** : an entity mapped to several table
- **@Embedded** : several classes (@Embeddable) mapped to a table

- **@Transient** : a non-persistent attribute
- **@Temporal** : date and time attribute
- **@Lob** : large objects types (CLOB, BLOB)
- **@ElementCollection** : collection of basic/embedded elements
- …

ENTITY RELATIONSHIP

JPA RELATIONSHIP ANNOTATIONS

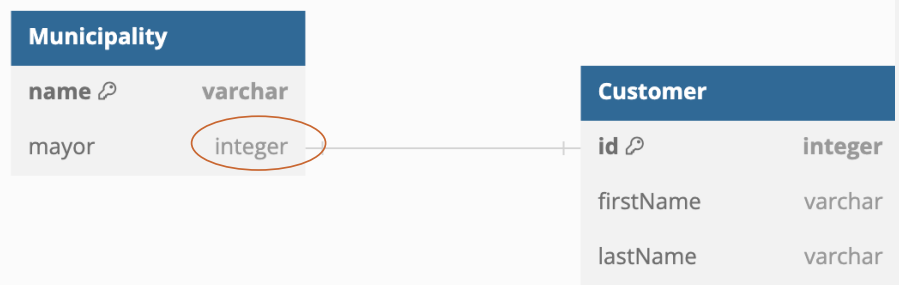
- **@OneToOne** - A unique reference from one object to another.
- **@OneToMany** - A Collection or Map of objects.
- **@ManyToOne** - A reference from one object to another, inverse of a OneToMany.
- **@ManyToMany** - A Collection or Map of objects.

1 - 1

```
@Entity
public class Municipality {
  @Id
  private String name;

  @OneToOne
  private Customer mayor;

  //...
```

Owner side

```
@Entity
public class Customer {
  @Id
  private Long id;

  @OneToOne(mappedBy = "mayor")
  private Municipality city;

  //...
```



Optional, only for navigability
Not persisted!

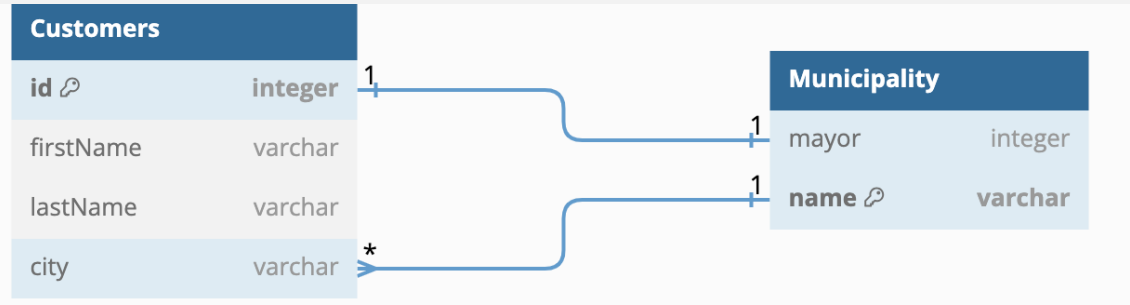
1-N

```java
public class Municipality {
  @Id
  private String name;

  @OneToMany(mappedBy = "city")
  private Collection<Customer> citizens;

  //...
```

```java
@Entity
public class Customer {

  @Id
  private Long id;

  @ManyToOne
  private Municipality city;
```

Owner side

Optional, only for navigability
Not persisted!

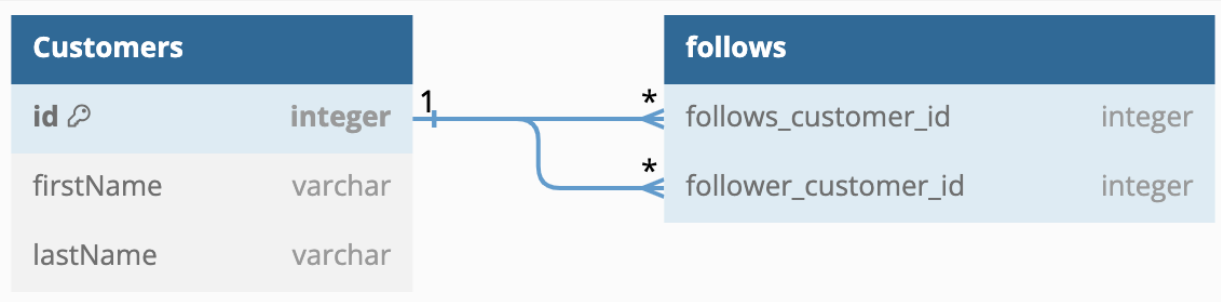


N-N

```
@Entity
public class Customer {

  @Id
  private Long id;

  @ManyToMany(mappedBy = "followers")
  Collection<Customer> follows;

  @ManyToMany
  Collection<Customer> followers;
```

Owner side

Optional, only for navigability
Not persisted!



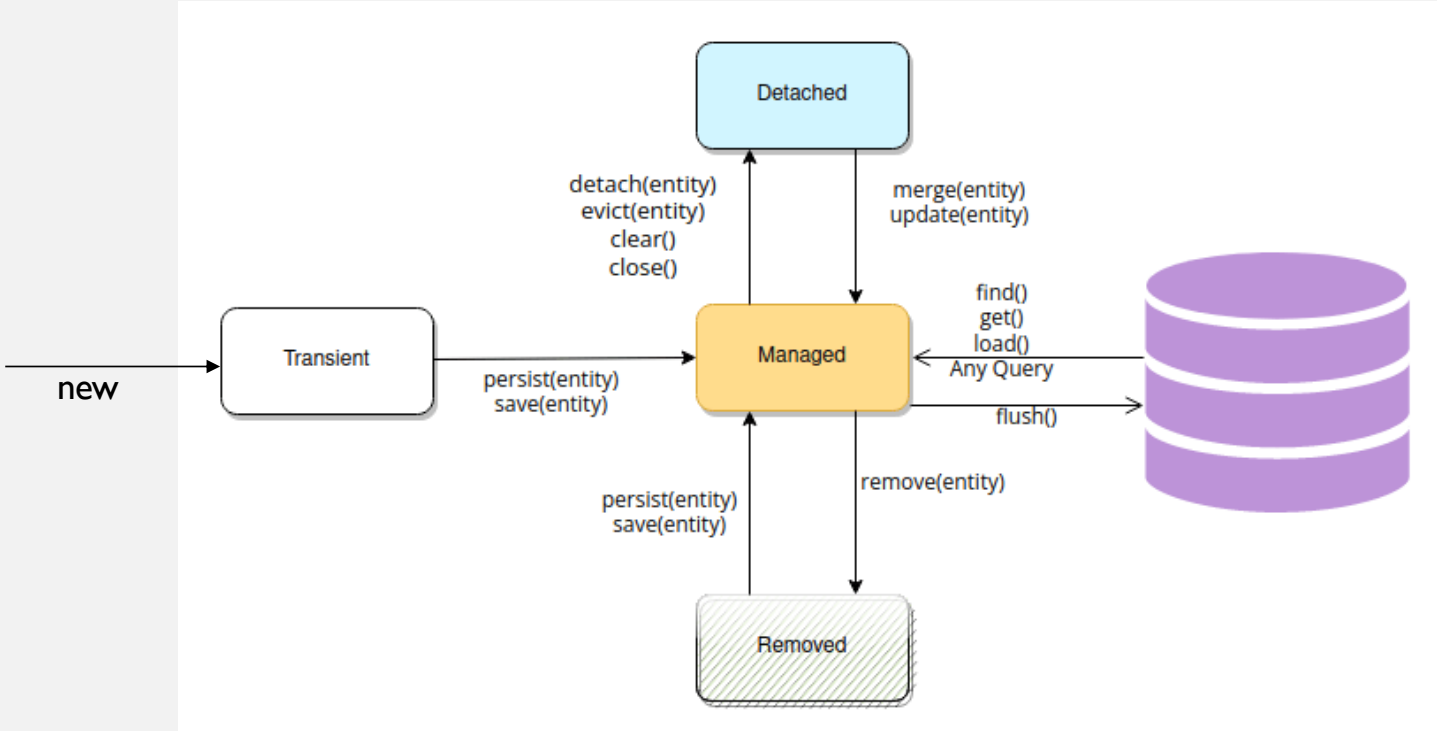
DB ANNOTATION

- **@JoinColumn** : Specifies the column for joining (foreign key 1-N)
- **@JoinTable** : Specifies the mapping of associations (N-N)
  - On the owning side
  - Example :

```
@ManyToMany
@JoinTable(name = "follows",
  joinColumns = @JoinColumn(name = "follower_customer_id"),
  inverseJoinColumns = @JoinColumn(name = "follows_customer_id"))
Collection<Customer> followers;
```

ENTITY MANAGEMENT

ENTITY LIFECYCLE



$Entity \neq Data$

ENTITY MANAGER

- In charge of managing entity lifecycle
- https://jakarta.ee/specifications/persistence/2.2/apidocs/javax/persistence/entitymanager
- Entity methods : persist, refresh, merge, remove, detach
- Find method : find(class, key)
- Query methods (JPQL)
- Transacations
- …

ENTITY MANAGER FACTORY

```
@PersistenceUnit
private EntityManagerFactory emf;
// ...
  EntityManager em = emf.createEntityManager();
  em.persist(new Customer("first", "last", new Municipality("city")));
  em.remove(em.find(Customer.class, 2L));
```

```
@PersistenceContext
private EntityManager em;
```

Non thread-safe

SPRING CRUD

- https://github.com/PascalUrso/CustomerJPADemo
- `mvn spring-boot:run`